Project Resource Allocation Platform – Solution Spec

Oct 12, 2026 · @Sigmund

Overview

Initiative owners submit projects with a business case and the FTEs they need per skill type. Approvers review them against available capacity and approve in full, in part, or reject. Reporting shows current, projected and unused FTE per skill and month.

Core design principle: FTE need is never stored as one number plus a duration. Every request is spread into one row per skill per month (time-phased). Current, projected and unused FTE then become simple aggregations.

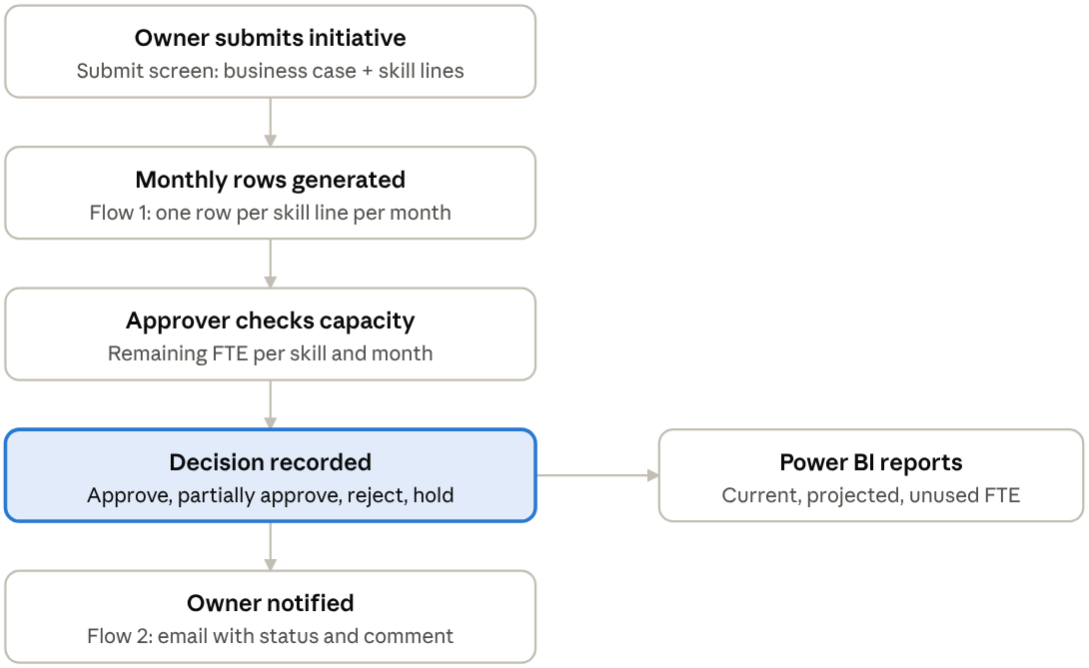
Stack

- **SharePoint:** five lists hold all data (SkillTypes, Initiatives, ResourceRequests, Allocations, Capacity)
- **Power Apps (canvas):** submit, follow up, and review with a per-skill capacity check
- **Power Automate:** generates the monthly allocation rows and sends notifications
- **Power BI:** reporting on current, projected, pipeline and unused FTE

Approval happens in the app, not the Approvals connector. Approvers need the capacity check and partial approval. The Approvals connector only offers Approve/Reject, and its wait times out after 30 days.

All components use standard connectors, so normal Microsoft 365 licences cover the solution. Dataverse or premium connectors would change the cost.

Capacity is checked per skill and month before any FTE is approved



request-to-report flow · 5 steps

Approved allocations and the Capacity list feed Power BI directly; nothing is reported from request totals.

SharePoint data model

Five lists. Initiatives are the header, ResourceRequests are the skill lines, and Allocations are the time-phased rows everything reports on. Index the columns marked as indexed before the lists grow.

SkillTypes (reference)

Column	Type	Notes
Title	Text	e.g. Developer, Data Analyst, Project Manager
IsActive	Yes/No	Retire skills without breaking history
SortOrder	Number	Dropdown ordering

Initiatives (one row per submission)

Column	Type	Notes
Title	Text	Initiative name
Description	Multi-line	
BusinessCase	Multi-line	Mandatory, minimum length enforced in the app
ExpectedBenefit	Multi-line	
StrategicFit	Choice	Organisation's categories
Owner	Person	
Department	Choice	
Priority	Choice	High / Medium / Low
StartDate / EndDate	Date	Earliest start and latest end across the request lines
Status	Choice	Submitted, Approved, Partially Approved, Rejected, On Hold, Active, Closed
DecisionBy	Person	
DecisionDate	Date	
DecisionComment	Multi-line	

ResourceRequests (one line per skill needed)

Column	Type	Notes
Title	Text	Set to the skill name, or make optional
InitiativeID	Number	Indexed. Plain number instead of a lookup: easier to patch and to join in Power BI
SkillType	Text	Matches SkillTypes.Title
FTE	Number	Decimals allowed (0.5 FTE)
StartDate / EndDate	Date	Per line, defaults to the initiative dates

Allocations (generated by flow: one row per request line per month)

Column	Type	Notes
InitiativeID	Number	Indexed
RequestID	Number	
SkillType	Text	Indexed
Month	Date	First of month. Used for filtering and in Power BI
MonthKey	Text	`2026-11`. Use this for joins, not the date
FTERequested	Number	
FTEApproved	Number	Approvers can lower this (partial approval)
Status	Text	Pending / Approved / Rejected. Text, not Choice, so filters stay simple and delegable. Written only by the app and flows

Capacity (maintained by the FTE pool owner)

Column	Type	Notes
SkillType	Text	Indexed
Month	Date	Indexed
MonthKey	Text	`2026-11`
AvailableFTE	Number	

Power Apps: screens and submit logic

Five screens. Wireframes for each are in the separate wireframe canvas.

Screen	Who	Purpose
Home	Everyone	Entry point: new initiative, my initiatives, review queue (approvers only)
Submit Initiative	Initiative owners	Business case, dates, and one line per skill type with FTE and period. Live FTE-month total

Screen	Who	Purpose
My Initiatives	Initiative owners	Status of own submissions, decision comments, approved vs requested FTE
Review Queue	Approvers	Pending initiatives sorted by priority and submission date
Review Detail	Approvers	Business case plus the per-skill, per-month capacity check. Approve, partially approve, reject, or put on hold

Show the Review Queue button only to approvers, for example by checking membership of an approver SharePoint group or a small Approvers list.

Submit screen: OnVisible

Seed the collection with one line so the schema exists:

```
ClearCollect(
    colSkillLines,
    { LineID: 1, SkillType: "", FTE: 0.0, StartDate: Today(), EndDate:
DateAdd(Today(), 3, TimeUnit.Months) }
)
```

Skill-line gallery (galSkillLines, Items = colSkillLines)

- `ddSkill.Items`: `SortByColumns(Filter(SkillTypes, IsActive = true), "SortOrder")`
- `ddSkill.OnChange`: `Patch(colSkillLines, ThisItem, { SkillType: Self.Selected.Title })`
- `txtFTE.OnChange`: `Patch(colSkillLines, ThisItem, { FTE: Value(Self.Text) })`
- `dpLineStart.OnChange`: `Patch(colSkillLines, ThisItem, { StartDate: Self.SelectedDate })` (same pattern for end date)
- Remove icon: `Remove(colSkillLines, ThisItem)`

Add-line button:

```
Collect(
    colSkillLines,
    { LineID: Max(colSkillLines, LineID) + 1, SkillType: "", FTE: 0.0,
      StartDate: Today(), EndDate: DateAdd(Today(), 3, TimeUnit.Months) }
)
```

Live total label:

```
"Total request: " &
Text(
    Sum(colSkillLines, FTE * (DateDiff(StartDate, EndDate, TimeUnit.Months) +
1)),
    "0.0"
) & " FTE-months"
```

Submit button: DisplayMode (validation)

```
If(
    IsBlank(txtTitle.Text)
    || Len(txtBusinessCase.Text) < 100
    || CountRows(colSkillLines) = 0
    || CountRows(Filter(colSkillLines, IsBlank(SkillType) || FTE <= 0 ||
EndDate < StartDate)) > 0,
    DisplayMode.Disabled,
    DisplayMode.Edit
)
```

Submit button: OnSelect

```
Set(
    varNewInit,
    Patch(
        Initiatives,
        Defaults(Initiatives),
        {
            Title: txtTitle.Text,
            Description: txtDescription.Text,
            BusinessCase: txtBusinessCase.Text,
            ExpectedBenefit: txtBenefit.Text,
            Priority: { Value: ddPriority.Selected.Value },
            Status: { Value: "Submitted" },
            StartDate: Min(colSkillLines, StartDate),
            EndDate: Max(colSkillLines, EndDate),
            Owner: {
                Claims: "i:0#.f|membership|" & Lower(User().Email),
                DisplayName: User().FullName,
                Email: User().Email,
                Department: "", JobTitle: "", Picture: ""
            }
        }
    )
```

```
);
Collect(
    ResourceRequests,
    ForAll(
        colSkillLines As L,
        {
            Title: L.SkillType,
            InitiativeID: varNewInit.ID,
            SkillType: L.SkillType,
            FTE: L.FTE,
            StartDate: L.StartDate,
            EndDate: L.EndDate
        }
    )
);
GenerateAllocations.Run(varNewInit.ID);
Notify("Initiative submitted", NotificationType.Success);
Navigate(scrMyInitiatives)
```

The flow is called from the app after the lines are saved. A "When an item is created" trigger on Initiatives can fire before the request lines exist and produce initiatives with no allocations.

Review screen: capacity check and decisions

The approver sees, per skill and month, the capacity, what is already approved, what this initiative asks for, and what remains. Red rows mean approving would over-allocate that skill in that month.

Load data when an initiative is selected (galPending.OnSelect)

```
Set(varInitID, ThisItem.ID);
ClearCollect(colReq, Filter(Allocations, InitiativeID = varInitID));
Set(varMin, Min(colReq, Month));
Set(varMax, Max(colReq, Month));
ClearCollect(colApproved, Filter(Allocations, Status = "Approved", Month >=
varMin, Month <= varMax));
ClearCollect(colCap, Filter(Capacity, Month >= varMin, Month <= varMax))
```

These filters are delegable (equals on text and number, range on dates). Sums and lookups then run locally on small collections, which avoids the 500/2000-row limits.

Capacity gallery: Items

```
ForAll(
    colReq As R,
    {
        ID: R.ID,
        MonthKey: R.MonthKey,
        SkillType: R.SkillType,
        Requested: R.FTERequested,
        Capacity: Coalesce(LookUp(colCap, SkillType = R.SkillType && MonthKey
= R.MonthKey, AvailableFTE), 0),
        AlreadyApproved: Sum(Filter(colApproved, SkillType = R.SkillType &&
MonthKey = R.MonthKey), FTEApproved)
    }
)
```

- Remaining label: `ThisItem.Capacity - ThisItem.AlreadyApproved - ThisItem.Requested`
- Row colour: `If(ThisItem.Capacity - ThisItem.AlreadyApproved - ThisItem.Requested < 0, Color.Red, Color.Green)`

Approve in full

```
Patch(
    Allocations,
    colReq,
    ForAll(colReq As R, { FTEApproved: R.FTERequested, Status: "Approved" })
);
Patch(
    Initiatives,
    LookUp(Initiatives, ID = varInitID),
    { Status: { Value: "Approved" }, DecisionDate: Today(), DecisionComment:
txtComment.Text }
)
```

Partial approval

Add an editable FTE input per row in the capacity gallery, defaulting to the requested FTE. Patch `FTEApproved` from that input instead of from `FTERequested`, and set the initiative status to Partially Approved.

Reject

Same pattern as approve, with `FTEApproved: 0, Status: "Rejected"` on the allocation rows and Status Rejected on the initiative. Require a decision comment before Reject is enabled.

Power Automate flows

Two flows: one turns request lines into monthly allocation rows, the other tells the submitter about decisions.

Flow 1: GenerateAllocations

Trigger: Power Apps (V2), one number input `InitiativeID`.

1. **Get items** on ResourceRequests, filter query `InitiativeID eq @{triggerBody()['number']}`
2. **Apply to each** request line:
   a. **Compose LocalStart:** `convertFromUtc(items('Apply_to_each')?['StartDate'], 'W. Europe Standard Time')`
   b. **Compose LocalEnd:** same for EndDate
   c. **Compose MonthCount:**
   `add(add(mul(sub(int(formatDateTime(outputs('LocalEnd'),'yyyy')), int(formatDateTime(outputs('LocalStart'),'yyyy'))), 12), sub(int(formatDateTime(outputs('LocalEnd'),'MM')), int(formatDateTime(outputs('LocalStart'),'MM')))), 1)`
   d. **Compose FirstMonth:** `startOfMonth(outputs('LocalStart'), 'yyyy-MM-dd')`
   e. **Apply to each (inner)** over `range(0, outputs('MonthCount'))`, then **Create item** in Allocations with the values below
3. **Send email or Teams message** to the approver group with a link to the Review Queue

Allocations column	Value
InitiativeID	Trigger input
RequestID	`items('Apply_to_each')?['ID']`
SkillType	`items('Apply_to_each')?['SkillType']`
Month	`addToTime(outputs('FirstMonth'), item(), 'Month', 'yyyy-MM-dd')`

Allocations column	Value
MonthKey	`addToTime(outputs('FirstMonth'), item(), 'Month', 'yyyy-MM')`
FTERequested	`items('Apply_to_each')?['FTE']`
FTEApproved	`0`
Status	`Pending`

Why convertFromUtc first: SharePoint stores dates in UTC. In Oslo, 1 November is stored as 31 October 22:00 or 23:00 UTC, so `startOfMonth` without conversion lands in October. The same reason is why joins use MonthKey instead of comparing dates.

Flow 2: Notify submitter on decision

Trigger: When an item is created or modified, on Initiatives, with trigger condition:

```
@not(equals(triggerOutputs()?['body/Status/Value'], 'Submitted'))
```

Action: email the Owner with the new status and DecisionComment, plus a link to My Initiatives.

To avoid repeat emails when other columns are edited later, add a hidden column `LastNotifiedStatus`. Compare it to Status in the trigger condition, and update it at the end of the flow.

Power BI reporting

Allocations and Capacity are both facts at skill × month grain. Connect them through a shared Date dimension (on Month) and a SkillType dimension. Never relate the two fact tables directly.

Measures

```
Capacity FTE  = SUM(Capacity[AvailableFTE])
Approved FTE  = CALCULATE(SUM(Allocations[FTEApproved]), Allocations[Status]
= "Approved")
Pipeline FTE  = CALCULATE(SUM(Allocations[FTERequested]), Allocations[Status]
= "Pending")
Unused FTE    = [Capacity FTE] - [Approved FTE]
Utilisation % = DIVIDE([Approved FTE], [Capacity FTE])
```

FTE is a per-month rate. Summing across a quarter gives FTE-months, not FTE. For quarter or year views, average over months, for example `AVERAGEX(VALUES('Date'[Month]), [Approved FTE])`.

Report views

Question	View
Current FTE use	Current month, Approved FTE by skill
Projected FTE use	Next 12–18 months: Approved and Pipeline as stacked bars, Capacity as a line, sliced by skill
Unused FTE	Matrix of skill × month showing Unused FTE, with negative values in red (over-allocation)
Initiative drill-down	Table of initiatives with requested vs approved FTE-months and status

Pitfalls and open decisions

Pitfalls

Pitfall	Mitigation
Delegation limits (500/2000 rows) give silently wrong results	Only delegable filters against SharePoint (equals, date ranges on indexed columns). Aggregate in local collections or Power BI
Time-zone shift moves dates into the previous month	`convertFromUtc` in flows, join on MonthKey text
Flow fires before request lines exist	Call the flow from the app after saving lines, not from a SharePoint trigger
Lookup columns are awkward in Patch and Power BI	Store plain number IDs
Allocations list grows fast (lines × months)	Index InitiativeID, SkillType, Month, Status from the start
Licensing creep	Stay on standard connectors and SharePoint. Dataverse or premium connectors change the cost

Open decisions

- [ ] Who owns the Capacity list, and how far ahead is it maintained? Suggested: a planner edits it in SharePoint grid view, 12 months ahead per skill
- [ ] What happens when an approved initiative changes (extended, more FTE)? Suggested: changes create new request lines that go through review again; existing allocations stay
- [ ] Single-step or multi-step approval (line manager, then portfolio board)?
- [ ] Final list of skill types
- [ ] Who are the approvers, and how is approver membership managed?